Step 2d — Formato del prompt: fisso, con il claim prima del documento

Obiettivo

Fissare il prompt di produzione. Come nel piano binario (../fine_tuning_factcheck/02d_varianti_domanda.md) la domanda è unica e congelata in `finetune/question.json`. Qui cambia una cosa: **l'ordine dei campi dello stato**.

1. Ordine claim → documento

Modello	Ordine nativo	Ordine di questo piano
Winnow-12B	`State: {"document":"…","claim":"…"}`	`{"claim":"…","document":"…"}`
JPT-9B	`document: …` / `claim: …`	`claim: …` / `document: …`

Il resto del prompt (system, domanda, opzioni, `Answer:`) è identico.

Perché: ogni frase del documento deve "leggere" il claim prima di sé, così il suo stato nascosto è già calcolato in funzione di ciò che si verifica (00a §4).

Conseguenze: - il verdetto del piano binario va **addestrato con questo ordine** (non si riusa un adapter addestrato con l'ordine nativo, salvo l'esperimento H4); - benchmark e baseline vanno rieseguiti con lo stesso ordine (B-ORD in 01).

2. Nessun marcatore nel documento

Il documento resta testo normale. Le frasi si individuano dagli offset (03), non con token inseriti come `[S1]`. Vantaggi: nessun cambio di formato oltre all'ordine; il testo che vede il modello è quello che vede l'utente.

3. Varianti

Nessuna variante di domanda né di nomi di campo: il chiamante è il nostro codice. Se in futuro l'input arriva con altri nomi (`context`, `answer`), si normalizzano prima del modello.

4. Attenzione con Winnow: il documento dentro il JSON

Nello stato di Winnow il documento è una stringa JSON: virgolette, a capo e < vengono **escapati** (`\"`, `\n`, `\u003c`). Gli offset delle frasi sul testo originale non coincidono con le posizioni nel prompt. Il rendering deve mappare ogni frase sul testo escapato (03 §3). Con JPT (righe `campo: valore`) il problema è minore ma va comunque gestito per gli a capo.

Criteri per passare allo step successivo

- `question.json` aggiornato con l'ordine dei campi per ciascun modello.
- B-ORD misurato.